AutoQuote — Cloud Run Deployment Runbook

Oct 20, 2026 · @Clark James

Deploying M Lhuillier AutoQuote takes one command and roughly 20 minutes, most of it spent waiting for Cloud SQL to come up. Work through the steps in order. Everything you need is inside the deployment package zip.

Before you start

Three things the package cannot supply:

1. **A Google Cloud project with billing enabled.** A new project is fine.
2. **Owner**, or Editor plus Project IAM Admin. The script creates service accounts and IAM bindings, which Editor alone cannot do.
3. **An authenticated `gcloud`.** Cloud Shell is already signed in, so there is nothing to do there.

You do **not** need Docker. Cloud Build builds the image in Google's infrastructure.

One thing worth checking first: if your organisation enforces a policy against public Cloud Run services or public buckets, the deploy will stop on that. Ask whoever administers the GCP organisation whether `constraints/iam.allowedPolicyMemberDomains` or a domain-restricted sharing policy applies to the project. That is a policy conversation, not something the package can work around.

Step 1 — Get the package onto the machine

Cloud Shell is the path of least resistance: `gcloud`, `openssl` and `cloud-sql-proxy` are already installed, and you are already authenticated.

1. Open console.cloud.google.com and select the target project.
2. Click the terminal icon in the top right to open Cloud Shell.
3. Use ⋮ → **Upload** and pick `ml-autoquote-cloudrun-20261020-1657.zip`, or drag it onto the terminal.
4. Unpack it and make the scripts executable:

```
unzip ml-autoquote-cloudrun-20261020-1657.zip -d autoquote
cd autoquote
chmod +x scripts/*.sh docker-entrypoint.sh
```

The `chmod` matters. Zip does not reliably preserve the executable bit, so the scripts often arrive non-executable and fail with "permission denied".

If you would rather work on your own machine, that works too — install the gcloud CLI, run `gcloud auth login`, then use `scripts/deploy.ps1` on Windows or `scripts/deploy.sh` on Linux and macOS. The two scripts do the same things.

Step 2 — Run the deploy

One command. Replace `YOUR_PROJECT_ID` with the real project id.

```
./scripts/deploy.sh --project YOUR_PROJECT_ID
```

On Windows PowerShell instead:

```
.\scripts\deploy.ps1 -ProjectId YOUR_PROJECT_ID
```

It is idempotent — every step checks what already exists — so the same command is both the first deploy and every one after.

What it does	Roughly
Enables the seven required APIs	1–2 min
Creates Artifact Registry, service account and IAM roles	under a minute
Creates the Cloud SQL MySQL instance	5–10 min
Creates the photo bucket and its CORS rules	under a minute
Generates and stores the two secrets	under a minute
Builds the container image in Cloud Build	3–5 min
Deploys the Cloud Run service	1–2 min
Applies the database schema as a one-off job	1–2 min

It prints the service URL when it finishes. Keep that — the rest of the steps use it.

The Cloud SQL step is the long one and gives little feedback. That is normal; leave it alone.

Run it a second time. On the very first deploy the public URL does not exist yet, so the canonical and Open Graph tags on the static pages are built with a placeholder. Once the service has an address, run it again to correct them:

```
./scripts/deploy.sh --project YOUR_PROJECT_ID --skip-infra
```

If you already have a custom domain, pass it the first time with `--site-url https://autoquote.mlhuillier.com` and you can skip the second run.

Step 3 — Create the first staff account

The deployed database has no users, so nobody can sign in until you make one.

```
./scripts/create-user.sh --project YOUR_PROJECT_ID \
  --email you@mlhuillier.com --name "Your Name" --role ADMIN
```

It opens a Cloud SQL Auth Proxy tunnel, creates the account, prints a generated password **once**, and closes the tunnel. Copy that password somewhere safe before you close the terminal.

Repeat for each appraiser, with `--role APPRAISER`.

Role	Can do
`ADMIN`	Everything an appraiser can, plus user management
`APPRAISER`	Review submissions, adjust offers, release them to customers
`VIEWER`	Read the queue, customer registry and dashboard only

Outside Cloud Shell you need the Cloud SQL Auth Proxy on your PATH. Cloud Shell already has it.

Step 4 — Check it worked

Two health endpoints, answering different questions.

```
curl https://YOUR-SERVICE-URL/api/health
curl https://YOUR-SERVICE-URL/api/health/ready
```

Endpoint	Answers	Expected
`/api/health`	Is the process serving? Never touches the database.	`{"status":"ok"}`
`/api/health/ready`	Is the database reachable and the schema applied?	`{"status":"ready","migrations":"applied"}`

If readiness returns `"migrations":"missing"`, the app is running but the schema was never applied. Run the migration job on its own:

```
./scripts/migrate-job.sh --project YOUR_PROJECT_ID
```

If you configure Cloud Run probes, point **startup and liveness at `/api/health`** and monitoring at `/api/health/ready`. Pointing a startup probe at readiness would stop a revision going live whenever Cloud SQL is cold, even though the public pages serve perfectly well without it.

Then open the site itself and click through:

- `https://YOUR-SERVICE-URL/` — the landing page
- `https://YOUR-SERVICE-URL/sell` — the four-step customer form
- `https://YOUR-SERVICE-URL/staff/login` — sign in with the account from Step 3

Step 5 — Connect SMS before customers use it

The platform texts each customer a link when their offer is released. Until a gateway is configured those messages go to Cloud Logging instead — fine for testing, useless in production, because the customer never learns their offer is ready.

```
gcloud run services update ml-autoquote --region=asia-southeast1 \
  --update-env-vars="SMS_WEBHOOK_URL=https://your-gateway/send"
```

If the gateway needs a token, store it as a secret rather than an environment variable:

```
printf 'YOUR_TOKEN' | gcloud secrets create autoquote-sms-token --data-file=-
gcloud run services update ml-autoquote --region=asia-southeast1 \
  --update-secrets="SMS_WEBHOOK_TOKEN=autoquote-sms-token:latest"
```

The platform POSTs `{"to": "09171234567", "message": "..."}` to that URL. Any gateway accepting a JSON POST will work.

To check what it would have sent before wiring anything up, release a test offer and read the logs:

```
gcloud run services logs read ml-autoquote --region=asia-southeast1 --
limit=50 | grep sms
```

Deploying again later

Once the infrastructure exists, skip it:

```
./scripts/deploy.sh --project YOUR_PROJECT_ID --skip-infra
```

That rebuilds the image, deploys a new revision and applies any pending migrations. Three to five minutes.

If the release changed `prisma/schema.prisma` and you want to apply the schema without redeploying:

```
./scripts/migrate-job.sh --project YOUR_PROJECT_ID
```

Cloud Run keeps previous revisions, so a bad release can be rolled back from the console under **Revisions → Manage traffic**, or:

```
gcloud run services update-traffic ml-autoquote --region=asia-southeast1 --
to-revisions=PREVIOUS_REVISION=100
```

Note that a rollback moves the application back but not the database. A migration that has already run stays run.

If something goes wrong

What you see	What it means	What to do
`permission denied` running a script	Zip dropped the executable bit	`chmod +x scripts/*.sh docker-entrypoint.sh`
`PERMISSION_DENIED` creating a service account	Your account lacks Project IAM Admin	Ask for Owner, or Editor plus Project IAM Admin
Deploy rejected on `--allow-unauthenticated`	An org policy forbids public Cloud Run services	Policy exception from the GCP org admin. Not fixable in the package
Bucket creation rejected	An org policy forbids public access settings	Same conversation. The bucket is private either way
Cloud SQL create fails on quota	New projects have low regional quota	Request more, or pick another region with `--region`
Build fails in `gcloud builds submit`	First real build of the image	Read the Cloud Build log it links. That is the error, not a mystery
Revision fails to start, port error	Something blocked startup	Should not happen — the container never waits on the database. Check `gcloud run services logs read`
`/api/health/ready` says `"migrations":"missing"`	Schema never applied	`./scripts/migrate-job.sh --project YOUR_PROJECT_ID`
`/api/health/ready` says `"database":"unreachable"`	Cloud SQL connection or password wrong	Check the service has `--add-cloudsql-instances` and that the `autoquote-database-url` secret matches the SQL user
Cannot sign in to the staff portal	No accounts exist yet	Step 3
Customers never receive their offer	No SMS gateway configured	Step 5

Logs for anything else:

```
gcloud run services logs read ml-autoquote --region=asia-southeast1 --
limit=100
gcloud run jobs executions list --job=ml-autoquote-migrate --region=asia-
southeast1
```

The package also contains `PACKAGE.md` with the full environment-variable reference and the manual `gcloud` sequence, should you prefer to run the steps by hand rather than through the script.